



It's Your Life

with

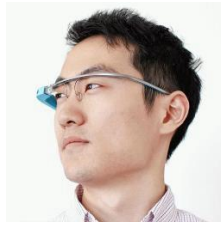




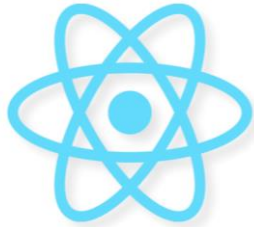
하는 일도 없이 허송세월하며
나이만 먹네요



facebook



Google



REACT





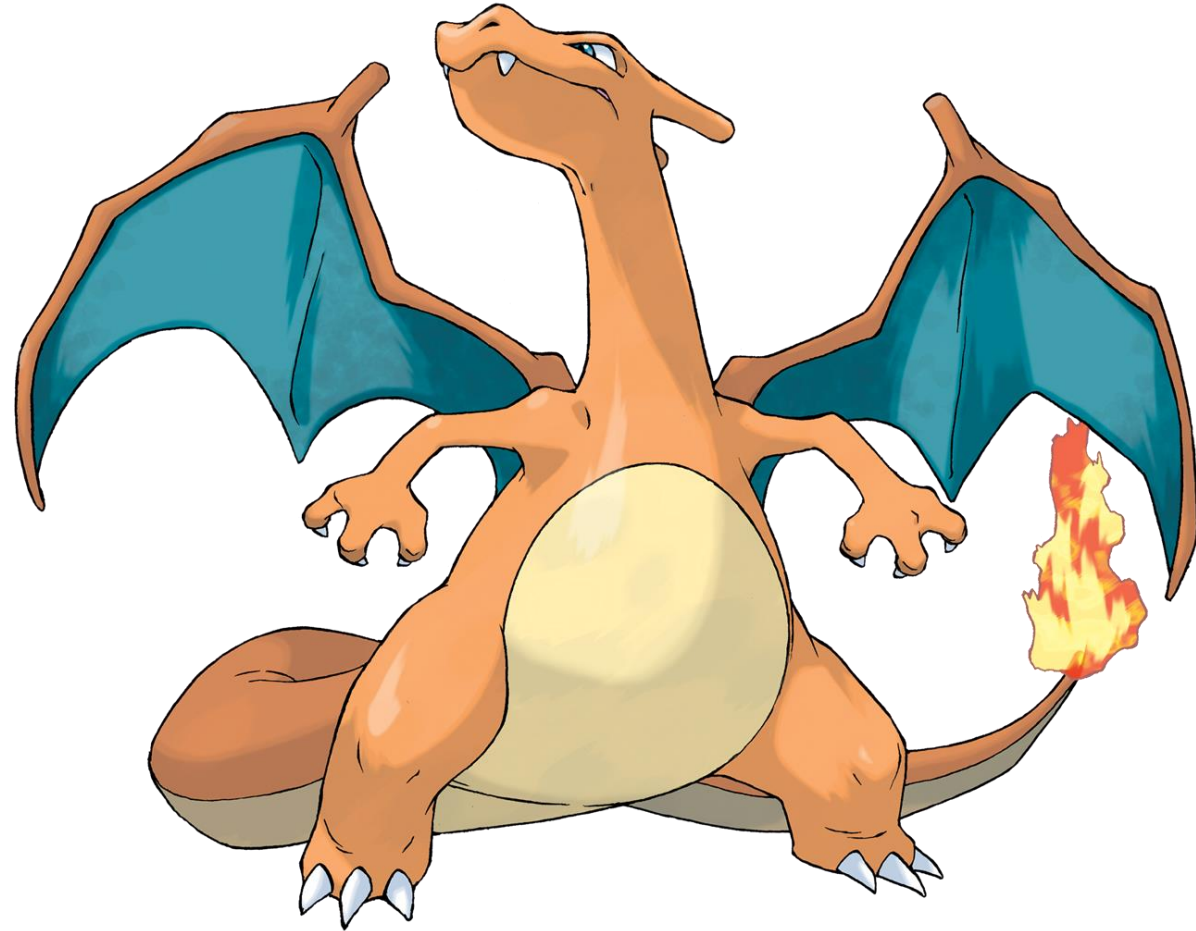
Vue 는 경쟁에서 살아남고자!!



- 최근 빠르게 변화하고 있는 프론트엔드 시장에서 Vue 는 살아남기 위해서 열심히 진화하고 있습니다!



V3





2007 스티븐 링 신장 올해의 베스트 동반
2007 IMDB가 선정 올해 최고의 영화 1위
IMDB 신장 관객이 뽑은 2000년 이후 최고의 영화 1위
미국 명문가 영화 신장 올해 최고의 영화 1위

나라는 없다

2008.02



Composition

API



이건 Vue2 와 Vue3 의 코드를 보시죠!



Vue2 문법

```
<template>
  <div>
    <h1>Vue2 문법을 사용</h1>
    <p>Count : {{ count }}</p>
    <p>두배로! {{ double }}</p>
    <button @click="increment">Increment</button>
  </div>
</template>
<script>
export default {
  data() {
    return {
      count: 0,
    };
  },
  computed: {
    double() {
      return this.count * 2;
    },
  },
  methods: {
    increment() {
      this.count++;
    },
  },
};
</script>
```

Vue3 문법

```
<template>
  <div>
    <h1>Vue3 문법을 사용</h1>
    <p>Count : {{ count }}</p>
    <p>두배로! {{ double }}</p>
    <button @click="increment">Increment</button>
  </div>
</template>

<script setup>
import { ref, computed } from 'vue';

const count = ref(0);
const double = computed(() => count.value * 2);

function increment() {
  count.value++;
}
</script>
```



또 배우고 외워야 하는
여러분의 표정.jpg



을 보는 강사



하지만

또 배우고, 외우셔야 합니다!



가수 #5353
김경호

가즈아!!!!!!!



data?

ref, reactive!!



한국 경제 진단과 처방

그때는 맞고 지금은 틀리다

도종록 | 민경신 | 범진완
송인창 | 정광조 | 정여진



Pride and Prejudice of Korean Economy

ref!?



- 과거에는 data 를 어떻게 선언했죠!?

```
export default {  
  data() {  
    return {  
      count: 0,  
    };  
  },  
};
```

- 하지만 이제는!? ref 를 사용해서
한 줄로 처리 할 수 있습니다!

```
<script setup>  
import { ref } from 'vue';  
  
const count = ref(0);  
</script>
```

ref!?



- 단, 값에 접근을 하기 위해서는 `.value`를 사용해야만 합니다!

```
const count = ref(0);  
console.log('count : ', count);  
console.log('count.value : ', count.value);
```

```
count : Vue3Component.vue:15  
  RefImpl {__v_isShallow: false, dep: undefined, __v_isRef:  
    true, _rawValue: 0, _value: 0} ⓘ  
    ▶ dep: Map(2) {ReactiveEffect => 1, ReactiveEffect => 1}  
      __v_isRef: true  
      __v_isShallow: false  
      _rawValue: 0  
      _value: 0  
      value: (...)  
      ▶ [[Prototype]]: Object  
count.value : 0 Vue3Component.vue:16
```

VALUE



reactive!?

- 과거에는 객체 data 를 어떻게 선언
했죠!?

```
<script>
export default {
  data() {
    return {
      state: {
        x: 0,
        y: 0,
        result: 0,
      },
    };
  },
};
</script>
```



- 하지만 이젠!? reactive 를
사용!

```
<script setup>
import { reactive } from 'vue';

const state = reactive({ x: 0, y: 0, result: 0 });
</script>
```



COMPOSITION API

VUE 3





ref? reactive?







아아아아아

이렇게 된 이상
최대한 쉽게 설명
한다!

할

할

오케이!!



27 (화)
08:18

KBS1

“지훈이 팔고 올게”
“안 돼, 재 안 팔아”



이정도면 중간아상이다.

ref



reactive



핵심은

자기가 가지고 있는 데이터가 반응형
VS
자기 자신이 반응형





코드로 보시죠!

인생은 실전이야



```
<template>
  <div>
    <h1>아아 꿈을 꾸었습니다. 제가 차은우가 되는 꿈을...</h1>
    <h1>Ref Data : {{ refData.name }}</h1>
    <h1>Reactive Data : {{ reactiveData.name }}</h1>
    <button v-on:click="changeRef">Ref Data 변경</button>
    <button v-on:click="changeReactive">Reactive Data 변경</button>
  </div>
</template>
```

Ref 와 Reactive 데이터를
객체로 선언!

```
<script setup>
import { ref, reactive } from 'vue';

const refData = ref({ name: '이효석' });
const reactiveData = reactive({ name: '이효석' });
```

```
const changeRef = () => {
  refData.value = { name: '차은우' };
};
```

```
const changeReactive = () => {
  reactiveData = { name: '차은우' };
};
</script>
```

각각의 메서드가 실행되면
선언된 데이터를
다른 객체로 변경!



Vite App

localhost:5173

Gmail

GDR

O365

ON

GitHub

NCL

Claude

ChatGPT

NAVER

Netlify

CT

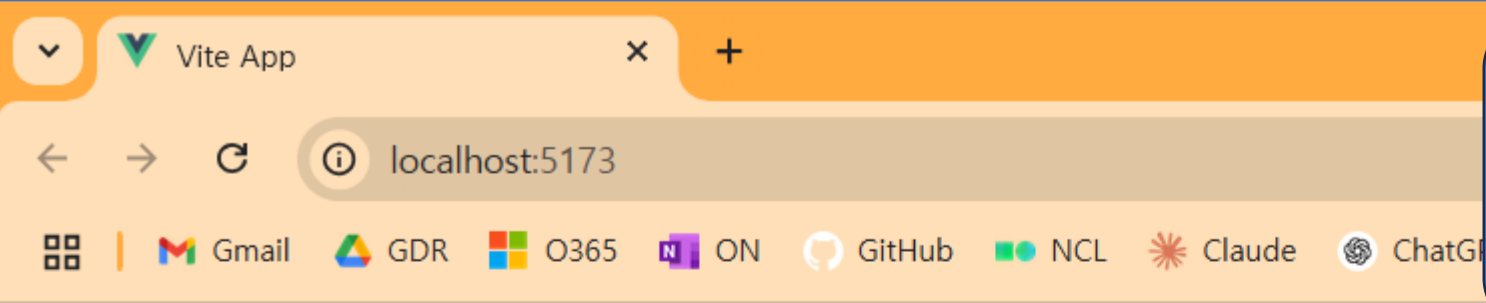
아아 꿈을 꾸었습니다. 제가 차은우가 되는 꿈을...

Ref Data : 차은우

Reactive Data : 이효석

Ref Data 변경

Reactive Data 변경



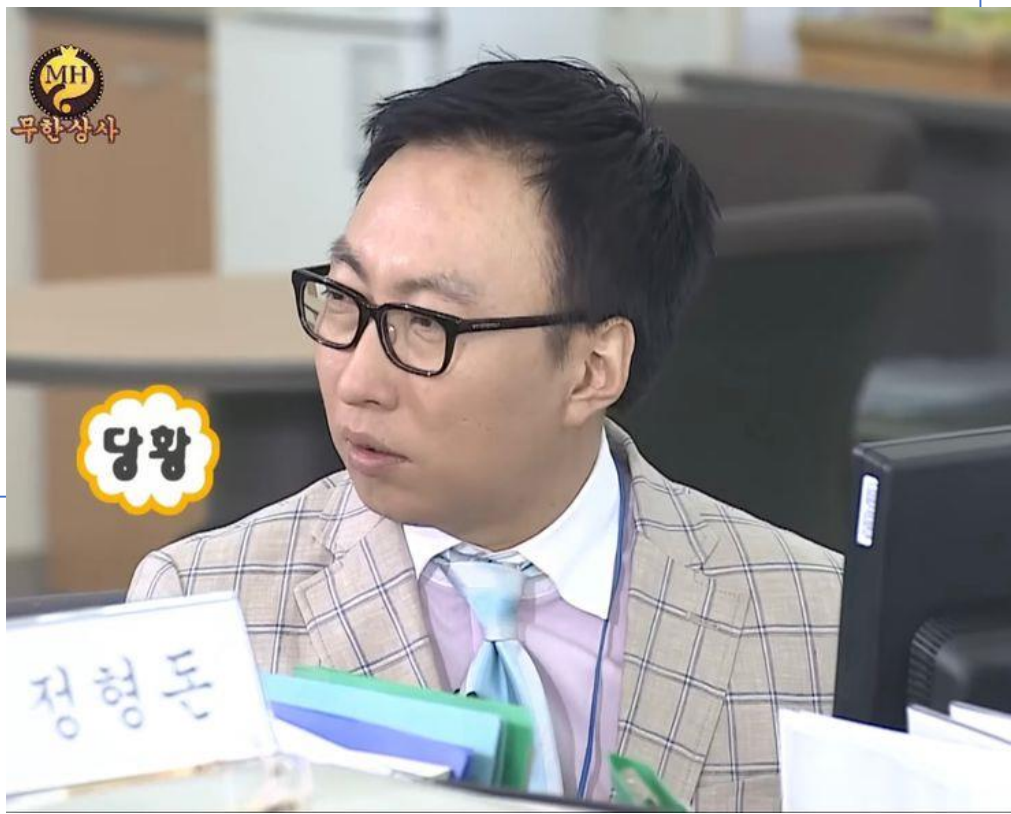
Reactive Data 는
아무리 클릭해도 변경 X

아아 꿈을 꾸었습니다. 제가 차은우가 되는 꿈을...

Ref Data : 차은우

Reactive Data : 이효석

Ref Data 변경 Reactive Data 변경



ref

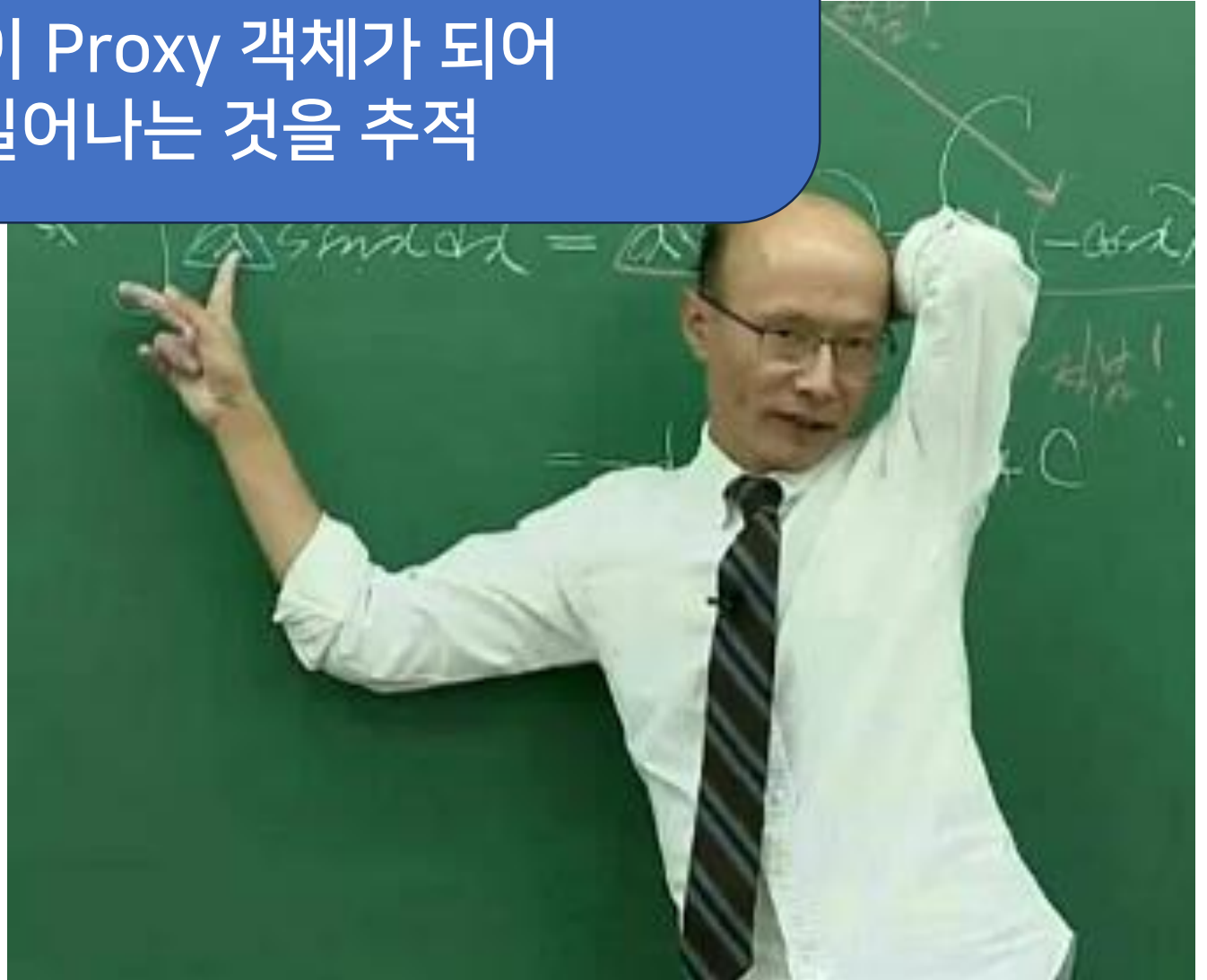


reactive



Ref 는 하나의 Proxy 객체로서
자기 내부의 value(장바구니) 의 값이 변경되는 걸 추적

Reactive 는 자기 자신이 Proxy 객체가 되어
자기 자신의 변화가 일어나는 것을 추적



ref



ref 는 장바구니라고 생각하면 편합니다!

자기가 담은 장바구니에 물품이 바뀌어도?

장바구니 자체가 변경되어도?

→ 사람은 알아 차릴 수 있습니다!

따라서, ref 는 객체 자체가 변경이 되어도
반응성이 유지가 됩니다!

ref 는 객체 내부에 getter, setter 를
가지고 있고 해당 함수가 호출 될 때
값 비교를 합니다!

reactive



reactive 는 자기를 보는 거울이라고
생각하면 편합니다!

자기 자신의 변화가 생기면?
→ 알아 차립니다!

그런데, 나 자신이 바뀌어 버리면?
→ 더 이상 거울을 볼 수 없기 때문에
변화 추적이 불가능 합니다!

reactive 는 객체의 주소를 바탕으로
변화를 추적하지만 새로운 객체로 변경되면
주소 값 자체가 바뀌기 때문에
reactive 자체를 잃어버리게 됩니다!



ref



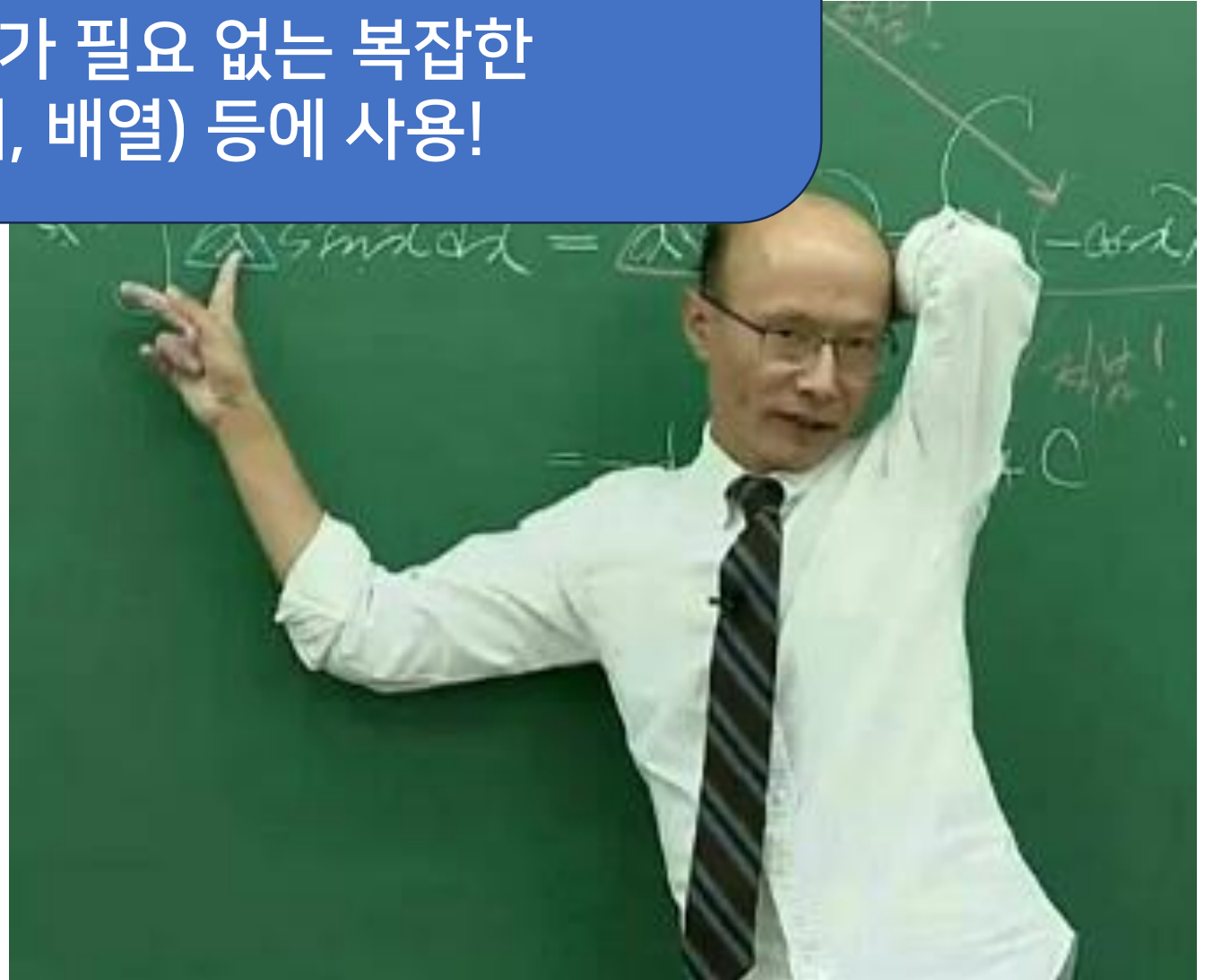
reactive



Ref 는 원시형 데이터 또는 나중에 전체 교체가 필요한
비교적 단순한 비원시형 데이터(객체, 배열) 등에 사용!



Reactive 는 전체 교체가 필요 없는 복잡한
비원시형 데이터(객체, 배열) 등에 사용!





아~ 완벽히 이해했어!



이해못했음



computed?

computed!

computed!?



- 과거에는 computed 를 어떻게 사용했죠?

```
<script>
export default {
  computed: {
    double() {
      return this.count * 2;
    },
  },
};
</script>
```

- 하지만 이제는!? computed
에 콜백 함수를 전달해서
바로 사용

```
<script setup>
import { computed } from 'vue';

const double = computed(() => count.value * 2);
</script>
```



computed 는 반응형
data 처럼 관리 됩니다!

* 아네! 아네! 잠깐만!

```
const count = ref(0);
const double = computed(() => count.value * 2);

function increment() {
  count.value++;

  console.log('🚀 ~ double:', double);
  console.log('🚀 ~ double.value:', double.value);
}
```

버튼을 클릭하면
computed 의 값을
한 번 확인해 봅시다!





~ double:

Coutner.vue:18

► ComputedRefImpl {setter: undefined, _value: 0, dep: Dep, __v_isRef: true, fn: f, ...}



~ double.value: 2

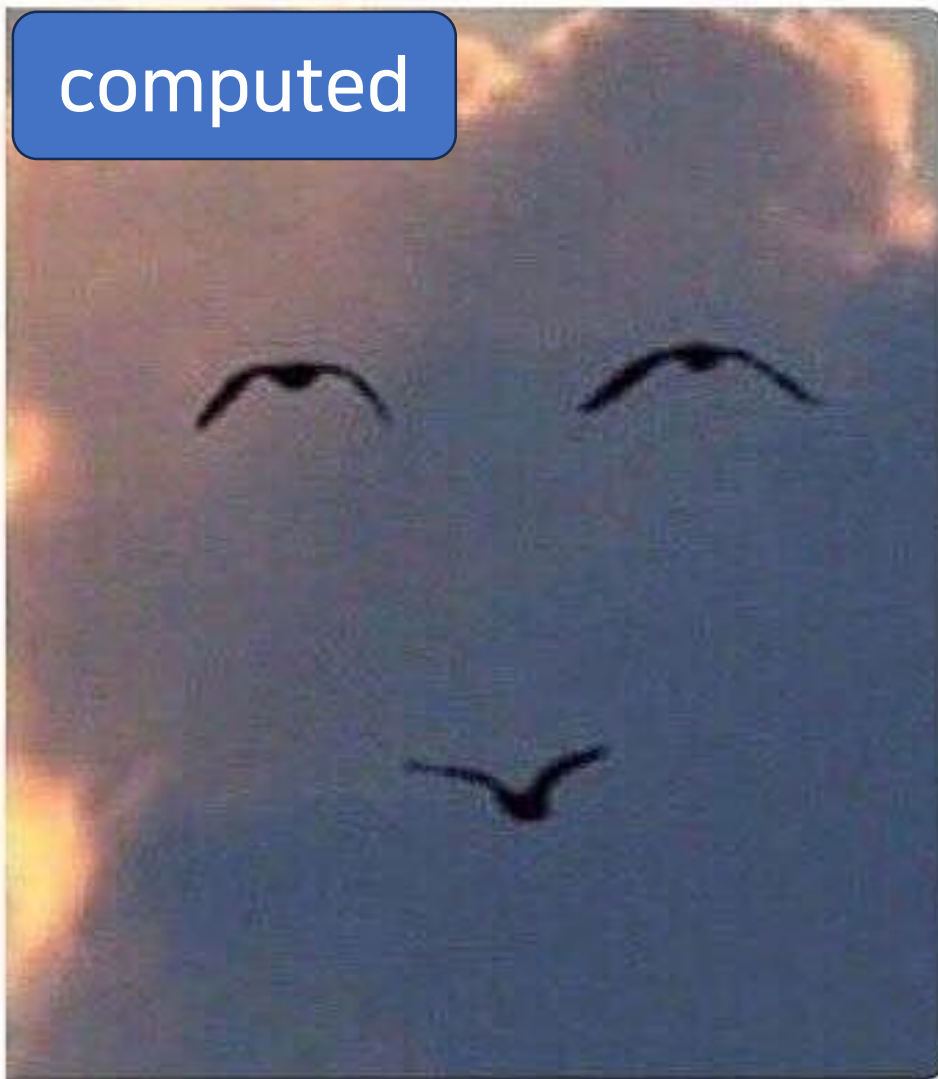
Coutner.vue:19



ref



computed





method?

그냥 함수로 !

method!?

- 과거에는 method 를 어떻게 사용했죠?



```
<script>
export default {
  methods: {
    increment() {
      this.count++;
    },
  },
};
</script>
```

- 하지만 이제는!? 그냥 함수를 선언해서
바로 사용!

```
<script setup>
function increment() {
  count.value++;
}
</script>
```



Counter

예제



```
<template>
  <div>
    <h1>Vue3 Counter</h1>
    <p>Count : {{ count }}</p>
    <p>두배로! {{ double }}</p>
    <button @click="increment">Increment</button>
  </div>
</template>
```

```
<script setup>
import { ref, computed } from 'vue';

const count = ref(0);
const double = computed(() => count.value * 2);

function increment() {
  count.value++;
}
</script>
```

<script setup> 을
활용하여 Vue3 의
CompositionAPI 사용



```
<template>
  <div>
    <h1>Vue3 Counter</h1>
    <p>Count : {{ count }}</p>
    <p>두배로! {{ double }}</p>
    <button @click="increment">Increment</button>
  </div>
</template>

<script setup>
import { ref, computed } from 'vue';

const count = ref(0);
const double = computed(() => count.value * 2);

function increment() {
  count.value++;
}
</script>
```

data 역할을 하는 ref 가져오기
computed 속성을 위한
computed 가져오기



```
<template>
  <div>
    <h1>Vue3 Counter</h1>
    <p>Count : {{ count }}</p>
    <p>두배로! {{ double }}</p>
    <button @click="increment">Increment</button>
  </div>
</template>
```

```
<script setup>
import { ref, computed } from 'vue';

const count = ref(0);
const double = computed(() => count.value * 2);

function increment() {
  count.value++;
}
</script>
```

ref 를 사용하여
data 역할을 하는
count 데이터 선언
및 0 으로 초기화

computed 의 콜백으로
computed 작업 설정
(주의) .value 로 접근



```
<template>
  <div>
    <h1>Vue3 Counter</h1>
    <p>Count : {{ count }}</p>
    <p>두배로! {{ double }}</p>
    <button @click="increment">Increment</button>
  </div>
</template>
```

```
<script setup>
import { ref, computed } from 'vue';

const count = ref(0);
const double = computed(() => count.value * 2);

function increment() {
  count.value++;
}
</script>
```

함수로 만든 메서드를 바로
이벤트에 걸어주기

함수 형태로 메서드 선언

실습, decrement 버튼 추가하기



- count 의 숫자를 1 감소시키는 decrement 버튼을 추가해 주세요
- increment 또는 decrement 가 수행 된 횟수를 기록하는 임의의 data 를 선언한 후, 수행 횟수를 화면에 출력하세요



Vue3 Counter

Count : 3

두배로! 6

지금까지 계산한 횟수 : 3

Increment

Decrement

Vue3 Counter

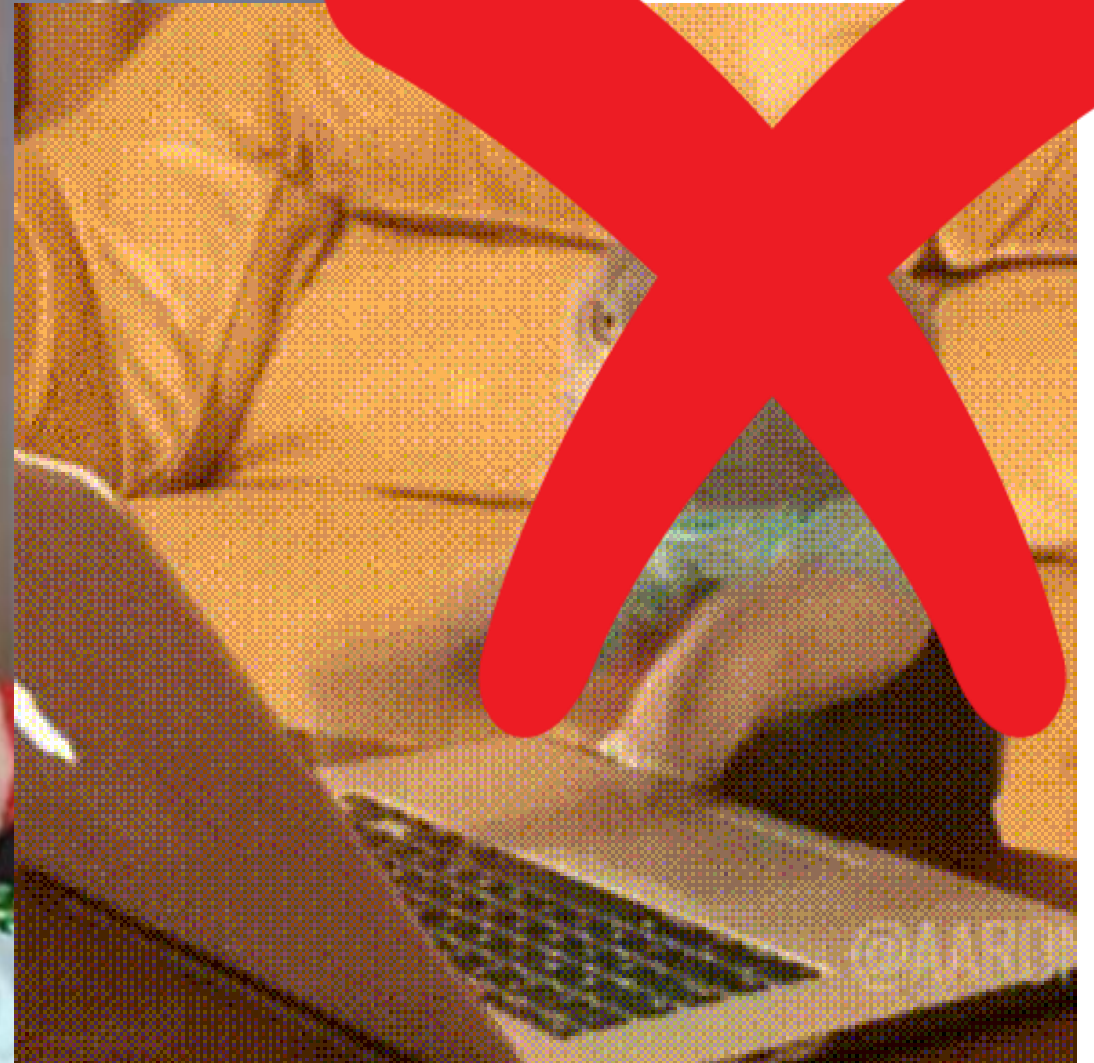
Count : -4

두배로! -8

지금까지 계산한 횟수 : 10

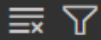
Increment

Decrement





vue



Vue 3 Snippets

A Vue.js 3 And Vue.js 2 Cod...
hollowtree



Vue VSCode Snippets

Snippets that will superchar...
sarah.drasner



Vue - Official

Language Support for Vue
Vue



vte

Syntax Highlight for Vue.js
jcuisson



Vue

Syntax Highlight
Rahul Kadyan

vte



Vue VSCode Snippets

sarah.drasner

3,072,583

★★★★★ (33)

Snippets that will supercharge your Vue workflow

Disable

Uninstall

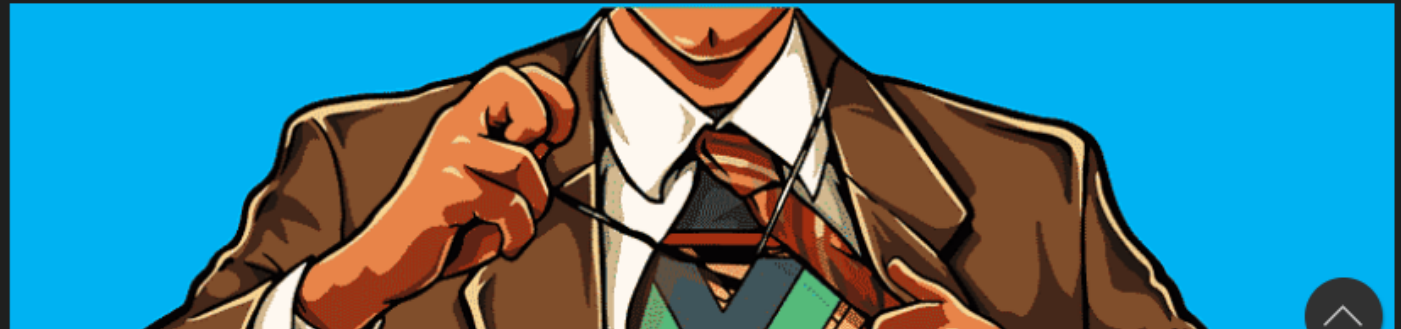
Auto Update

DETAILS

FEATURES

CHANGELOG

Vue VSCode Snippets



Vue3 문법에 대한
스니펫을 제공합니다!



▼ TestComponent.vue U ●

test > 07_VUE_APP > src > components > ▼ TestComponent.vue > ...

1 vbas

- ☐ vbase Vue Single File Component
- ☐ vbase-3 Vue Single File Component Composi...
- ☐ vbase-3-reacti... Vue Single File Component C...
- ☒ vbase-3-setup Vue Single File Component Setu...
- ☐ vbase-3-ts Vue Single File Component Composi...

vbase-3-setup 을 선택!

▼ TestComponent.vue U ●

test > 07_VUE_APP > src > components > ▼ TestComponer

```
1  <template>
2    <div>
3
4    </div>
5  </template>
6
7  <script setup>
8
9  </script>
10
11  <style lang="scss" scoped>
12
13  </style>|
```





watch?

watch!

watch?



- 과거에는 watch 를 어떻게 사용했죠?

```
<script>
export default {
  watch: {
    selectedCategory(changeCategory) {
      this.fetchFoodsByCategory(changeCategory);
    },
  },
};
</script>
```

watch!



- computed 마찬가지로 메서드를 임포트한 다음, 값과 콜백 함수를 전달해서 사용합니다!

watch 메서드에
관찰할 값, 콜백 함수를 전달해서
값의 변화에 따라 부수 작업을
처리하면 됩니다~!

```
<script setup>
import { ref, computed, watch } from 'vue';

watch(count, (newValue, oldValue) => {
  console.log(`카운트가 ${oldValue}에서 ${newValue}로 변경되었습니다.`);

  if (newValue % 10 === 0 && newValue !== 0) {
    alert(`축하합니다! ${newValue}까지 도달했습니다!`);
  }
});
</script>
```

localhost:5173 내용:

축하합니다! 10까지 도달했습니다!

확인





props?

defineProps!

??? :하늘에서 정의가 으아아아



Props 도 이제
정의해서 씁니다!



props!?



- 과거에는 props 를 어떻게 사용했죠?

```
<template>
  <div>
    <Child :drilling="drilling" />
  </div>
</template>
```

```
<script>
export default {
  name: 'App',
  data() {
    return {
      drilling: '으아아아아',
    };
  },
};
</script>
```

```
<template>
  <div>
    <h1>드디어 도착한 props 의 내용은!?!</h1>
    <h2>{{ drilling }}</h2>
  </div>
</template>
```

```
<script>
export default {
  name: 'GrandChild',
  props: ['drilling'],
};
</script>
```



- 이제는 defineProps 를 사용하여 정의하여 사용합니다!

```
<template>
  <div><Vue3Props :msg="msg" /></div>
</template>

<script setup>
import Vue3Props from
'./components/Vue3Props.vue';
import { ref } from 'vue';

const msg = ref('props 로 전달');
</script>
```

props 로 전달

```
<template>
  <div>
    <h1>{{ msg }}</h1>
  </div>
</template>

<script setup>
import { defineProps } from 'vue';

// props를 정의하고 기본값과 타입을 설정
const props = defineProps({
  msg: {
    type: String,
    required: true,
  },
});
</script>
```



emit?

defineEmits!



Emit 도 발산이 아니고
정의해서 씁니다!

← 발 산 (에스앤유서울병원)

Balsan
(SNU Seoul Hospital)
钵山 パルサン

emit!?



- 과거에는 emit 을 어떻게 사용했죠?



```
<template>
  <div>
    <input type="text" v-model.trim="name" />
    <button @click="$emit('name-changed', { name })">이름 전달!</button>
  </div>
</template>

<script>
export default {
  name: 'SendNameComponet',
  data() {
    return {
      name: '',
    };
  },
};
</script>
```

emit 을 보내는 자식 컴포넌트



```
<template>
  <div>
    <h1>너의 이름은 {{ name }}</h1>
    <SendNameComponent @name-changed="nameChangeHandler" />
  </div>
</template>
```

```
<script>
export default {
  name: 'App',
  data() {
    return {
      name: '',
    };
  },
  methods: {
    nameChangeHandler(e) {
      this.name = e.name;
    },
  },
};
</script>
```

emit 을 받는 부모 컴포넌트

emit!?



- Vue3 에서 바뀐 Emit 은!?


```
<template>
  <div>
    <input type="text" v-model.trim="inputMsg" />
    <button @click="sendMsg">부모님께 전송</button>
  </div>
</template>
```

```
<script setup>
import { ref, defineEmits } from 'vue';
const inputMsg = ref('');
const emit = defineEmits(['send-msg']);
```

```
function sendMsg() {
  emit('send-msg', inputMsg.value);
}
</script>
```

defineEmit 으로 사용 할
emit 을 배열에 선언

emit 을 보내는 자식 컴포넌트

배열에 선언 된 emit 이 전달할 데이터를 연결



```
<template>
  <div>
    <h1>{{ msg }}</h1>
    <EmitChild @send-msg="emitHandler" />
  </div>
</template>
```

전달한 emit 의 이름과 동일한 이벤트를
v-on 으로 연결!

```
<script setup>
import EmitChild from './components/EmitChild.vue';
import { ref } from 'vue';

const msg = ref('자식놈이 바꿀 메세지입니다!');

function emitHandler(value) {
  msg.value = value;
}
</script>
```

emit 을 받는 부모 컴포넌트

emit 이 전달한 data 를 매개변수로 받아서
필요한 작업을 처리!

실습, props 와 emit, computed, watch 사용

- 부모 컴포넌트(QuizParent.vue)에서 0 ~ 20 사이의 숫자를 랜덤으로 생성해서 자식컴포넌트(QuizChild.vue)로 props 를 통해 내려주세요
- 자식 컴포넌트는 input 요소가 있고, 정답 버튼이 있으며 부모로 부터 받은 랜덤 숫자를 맞출 수 있습니다
- 정답 버튼을 클릭하면 부모로 부터 받은 랜덤 값과 비교하여 상황에 따라 Up / Down / 정답입니다 를 emit 으로 전달 후, 부모 컴포넌트에서 출력하는 어플리케이션을 작성하세요
- watch 를 사용하여 정답 시도가 3번이 되는 순간 alert 으로 3번째 시도입니다! 를 출력해 주세요~!



랜덤 숫자를 맞춰 주세요

정답!

UP

정답!

DOWN

정답!

정답입니다!

정답!

localhost:5173 내용:

3번째 시도 입니다!

확인

Elements

Console

Sources